

Symulator wzmacniacza gitarowego - testowanie funkcjonalności aplikacji

Aleksander Gołębiewski 331378

January 20, 2025

Contents

1	Wstęp	2
2	Testy funkcjonalności - generowanie dźwięku	2
2.1	Dostosowywanie głośności pokrętłem Volume	2
2.2	Modyfikacja spektrum częstotliwości za pomocą korektora	6
2.2.1	Informacje dotyczące zaimplementowanego korektora	6
2.2.2	Testy klasy Filter	6
2.2.3	Testy klasy LiveEQ	9
2.3	Zmiana głośności za pomocą Gain	10
3	Testy funkcjonalności - efekty dźwiękowe	12
3.1	Klasa Delay	12
3.1.1	Krótki opis klasy Delay	12
3.1.2	Testowanie Delay	12
3.2	Klasa Flanger	16
3.2.1	Krótki opis klasy Flanger	16
3.2.2	Testowanie Flanger	16
3.3	Klasa Reverb	18
3.3.1	Krótki opis klasy Reverb	18
3.3.2	Testowanie Reverb	19

1 Wstęp

Niniejsza praca dotyczyć będzie testów funkcjonalności symulatora wzmacniacza gitarowego. Raport ten nie będzie dotyczyć kwestii sposobu użycia aplikacji, i nie stanowi on części instrukcji. W celu zapoznania się z działaniem aplikacji wzmacniacza gitarowego, należy zapoznać się z dostępną do aplikacji instrukcją.

Testy funkcjonalności będą dotyczyć działania poszczególnych modułów aplikacji, i będą one sprawdzać, czy oczekiwane rezultaty faktycznie są osiągane przez program. W spisie treści na **pierwszej stronie** widoczny jest podział testów na poszczególne sekcje.

Dodatkowo, do raportu dołączone zostaną pliki audio, które będą przywoływane w odpowiednich częściach sprawozdania jako punkt odniesienia. Oprócz tego dostępne też będą załączone w raporcie różne wizualizacje, jak i fragmenty kodu odpowiedzialne za funkcjonalność pewnych modułów. Opisywane moduły będą dotyczyć głównie generowania dźwięku, jak i nakładania efektów w czasie rzeczywistym.

2 Testy funkcjonalności - generowanie dźwięku

2.1 Dostosowywanie głośności pokrętką Volume

Testy będą przeprowadzać na dźwięku wygenerowanym **przez oscylator „square wave”**, generujący falę podobną do fali prostokątnej symetrycznej (lecz nie w pełni idealnej, aczkolwiek dla testów nie ma to aktualnie znaczenia). Obie nuty zostały zagrane z równą wartością **velocity=127**:

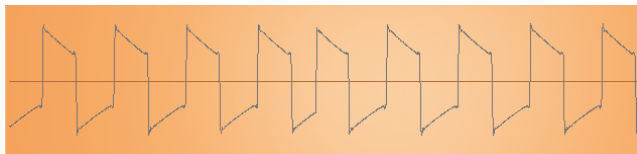


Figure 1: Fala dźwięku wygenerowana przez oscyloskop

Dźwięki używane w poniższym teście to **squareC_volume10.wav**, **squareC_volume05.wav** oraz **squareC_volume01.wav**. Pliki te zostały wygenerowane dla ustawień wzmacniacza **odpowiednio od lewej do prawej**:

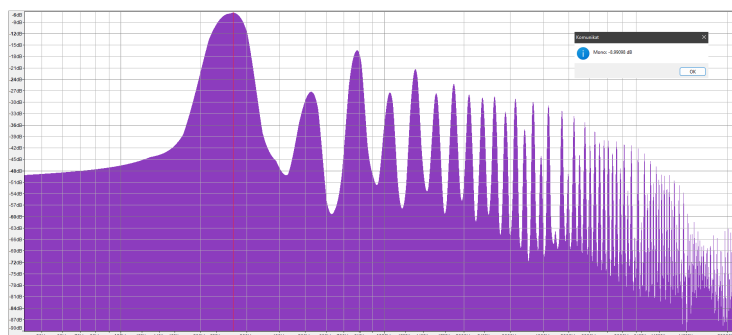


(a) Volume = 1.0

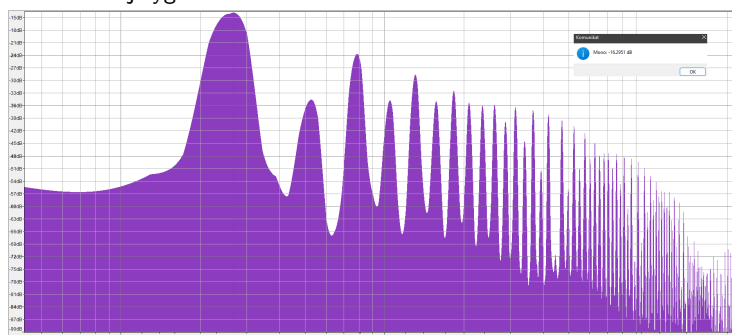
(b) Volume = 0.5

Figure 2: Ustawienia wzmacniacza do testów

Dla podanych ustawień wygenerowane zostały następujące wyniki:



(a) Volume = 1.0 - widmo częstotliwościowe oraz wartość średniej kwadratowej sygnału



(b) Volume = 0.5 - widmo częstotliwościowe oraz wartość średniej kwadratowej sygnału

Figure 3: Widma częstotliwościowe oraz wartości RMS

Na co zwrócić uwagę w analizie testów?

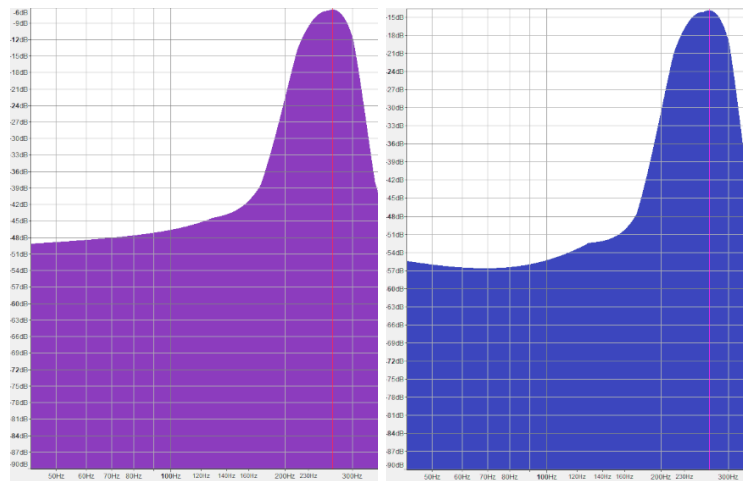
- Aby wykazać poprawność działania pokrętła Volume, można zwrócić uwagę na głośność częstotliwości fundamentalnej. Na obrazku (a) zauważyć można, że głośność częstotliwości fundamentalnej znajduje się w zakresie (-7dB; -6dB). **Częstotliwość fundamentalna dodatkowo oznaczona jest czerwonym odcinkiem.** Dla uproszczenia obliczeń założę, że głośność ta jest równa około -6.5 dB. W związku z tym, że głośność w relacji pomiędzy volume a decybelami jest logarytmiczna, nowa głośność powinna wynosić:

$$\Delta L = 20 \cdot \log_{10} \frac{v_n}{v_p}$$

$$\Delta L = 20 \cdot \log_{10} \frac{0.5}{1.0} \approx 20 \cdot (-0.301) = -6.02 [dB]$$

$$\Delta L = L_n - L_p \Rightarrow L_n = \Delta L + L_p \Rightarrow L_n = -6.02 + -6.5 = -12.52 [dB]$$

, gdzie L_n, L_p to głośność końcowa i głośność początkowa, ΔL to zmiana głośności, a v_n, v_p to wartość volume odpowiednio końcowa oraz początkowa.



(a) Volume = 1.0 - częstotliwość fundamentalna = 261.626 Hz (nuta C4)
(b) Volume = 0.5 - częstotliwość fundamentalna = 261.626 Hz (nuta C4)

Figure 4: Zmiana głośności częstotliwości fundamentalnej

Jak widać, wartość głośności fundamentalnej częstotliwości jest zgodna z oczekiwaniami. Widać, że głos jest cichszy, a działanie tego pokrętła **jest w pełni przewidywalne**

- Na rysunkach z pełnym widmem częstotliwości (3a i 3b) widnieją w prawym górnym rogu wartości RMS. W przypadku **Volume = 1.0, RMS = -8.99098 db**, a w przypadku **Volume = 0.5, RMS = -16.2951**.

W ramach dokończenia testów i wykazania poprawnego, zgodnego z oczekiwaniami działania pokrętła Volume, nagram jeszcze jeden dźwięk - tym razem dla Volume około 0.1. Obliczę najpierw głośność częstotliwości fundamentalnej 261.626 Hz:

$$\Delta L = 20 \cdot \log_{10} \frac{v_n}{v_p}$$

$$\Delta L = 20 \cdot \log_{10} \frac{0.1}{1.0} \approx 20 \cdot (-1) = -20 [dB]$$

$$L_n = \Delta L + L_p \Rightarrow L_n = -20 + -6.5 = -26.5 [dB]$$

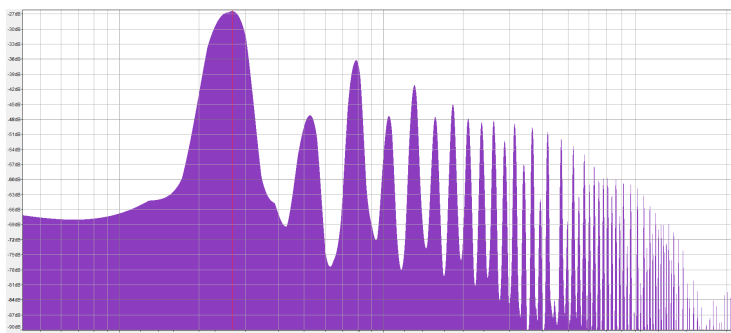


Figure 5: Volume = 0.1 - widmo częstotliwościowe

Z obrazka ponownie można odczytać wartość częstotliwości fundamentalnej oznaczonej czerwonym odcinkiem. Widać teraz jak poprawne są wyniki przy ułożeniu volume = 0.1 - głośność jest mniej więcej równa obliczonej wartości.



Figure 6: Ustawienie wzmacniacza do trzeciego testu głośności

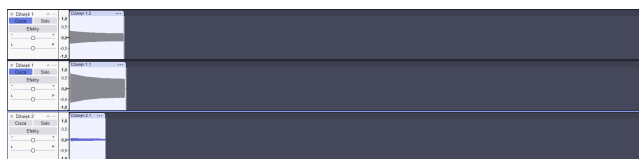


Figure 7: Projekt w Audacity, w którym generowane były wyniki

2.2 Modyfikacja spektrum częstotliwości za pomocą korektora

2.2.1 Informacje dotyczące zaimplementowanego korektora

Korektor LiveEQ w zasadzie w większości bazuje na **trzech filtrach typu Peak**, które na bazie zdefiniowanej środkowej częstotliwości zwiększają głośność pewnego zdefiniowanego zakresu częstotliwości liczonego jako $[f_0 - zakres; f_0 + zakres]$, gdzie f_0 to częstotliwość środkowa filtra. Aby więc sprawdzić poprawność działania korektora w czasie rzeczywistym, najpierw sprawdzę działanie filtrów:

Część kodu korektora inicjalizująca trzy obiekty klasy Filter + processSample

```
public class LiveEq {
    private final Filter lowFilter;
    private final Filter midFilter;
    private final Filter highFilter;

    LiveEq() {
        lowFilter = new Filter(44100f);
        midFilter = new Filter(44100f);
        highFilter = new Filter(44100f);

        lowFilter.setParameters(100.0f, 80.0f, 1.0f);
        midFilter.setParameters(800.0f, 400.0f, 1.0f);
        highFilter.setParameters(6000.0f, 2000f, 1.0f);
    }

    public float processSample(float sample) {
        float low = lowFilter.update(sample);
        float mid = midFilter.update(low);
        float high = highFilter.update(mid);
        return high;
    }
}
```

2.2.2 Testy klasy Filter

W poniższych testach będę generować nowe dźwięki. Będę je przywoływać w odpowiednich częściach testów.

Przebieg testów klasy Filter będzie następujący:

1. Wygenerowanie dźwięku dla pewnej częstotliwości z użyciem oscylatora fal sinusoidalnych
2. Próba podwyższenia częstotliwości dźwięku testowego o pewną wartość gain z minimalnym zakresem
3. Sprawdzenie wyników i porównanie widm częstotliwościowych
4. Próba podwyższenia zakresu częstotliwości dźwięku testowego o pewną wartość gain
5. Sprawdzenie widma oraz analiza wyników

Wygenerowałem dzięki stronie <https://onlinetonegenerator.com/> ton o częstotliwości fundamentalnej = 620 Hz. Ton znajduje się w pliku **620hz.wav**. Teraz zmienię nieco kod aplikacji tak, by **LowFilter** był w częstotliwości 620 Hz, i odpowiadał za gain w zasadzie jedynie tej częstotliwości (oraz niedużej okolicy):

```
LiveEq() {  
    lowFilter = new Filter(44100f);  
    midFilter = new Filter(44100f);  
    highFilter = new Filter(44100f);  
  
    lowFilter.setParameters(100.0f, 80.0f, 1.0f);  
    midFilter.setParameters(620.0f, 12.0f, 1.0f);  
    highFilter.setParameters(6000.0f, 2000f, 1.0f);  
}
```

Punkt drugi przebiegu testu zrealizuję, grając losową nutę na syntezatorze, i mocno zwiększając gain częstotliwości 620 Hz podczas gdy gra faktyczny dźwięk oscylatora odpowiadający tej częstotliwości. **Ten dźwięk można znaleźć w pliku test_filter620hz.wav.**

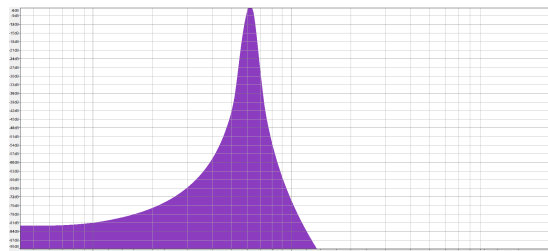


Figure 8: Widmo częstotliwościowe dla oscylatora fali sinusoidalnej o częstotliwości 620Hz

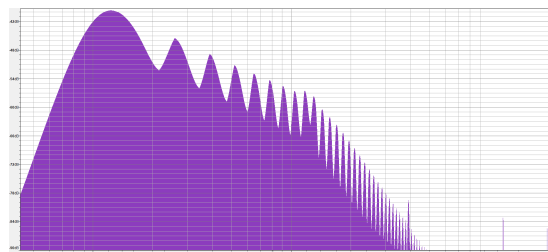


Figure 9: Widmo dźwięku testowego bez filtra

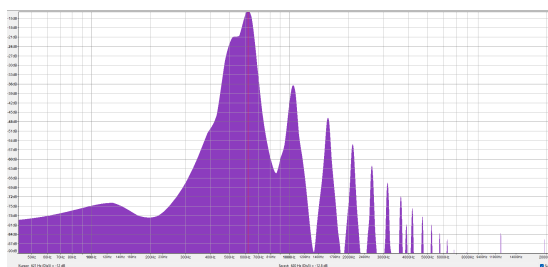


Figure 10: Widmo dźwięku testowego z filtrem peak na 620 hz

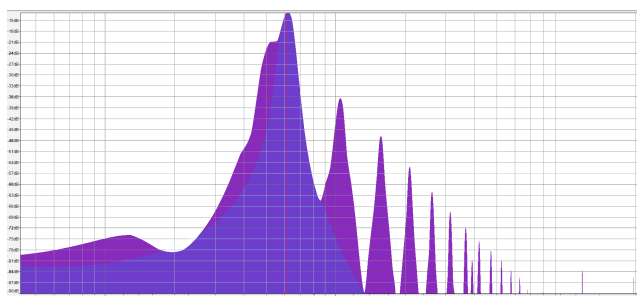


Figure 11: Zestawienie obu widm, częstotliwości pokrywają się.

Widać, że korektor zgodnie z przewidywaniami zwiększa poprawnie głośność tej pojedynczej, zdefiniowanej w teście częstotliwości. Teraz zwiększę częstotliwości w zakresie „high” - od 10000 Hz, do 20000 Hz.

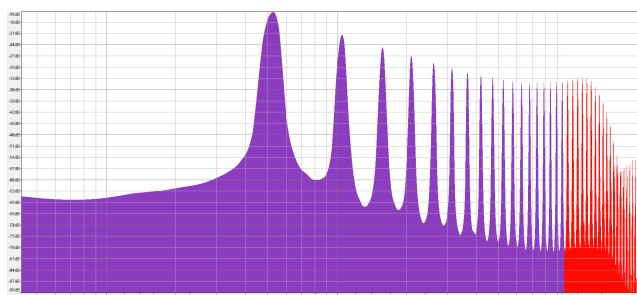


Figure 12: Widmo pierwotne nowego dźwięku testowego

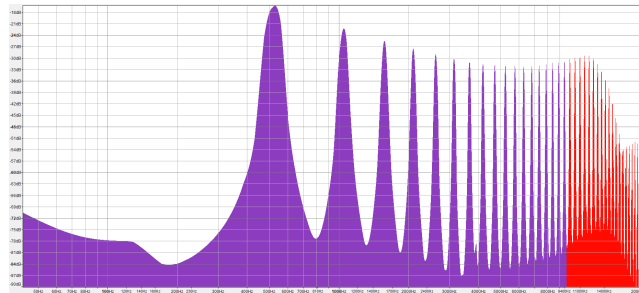
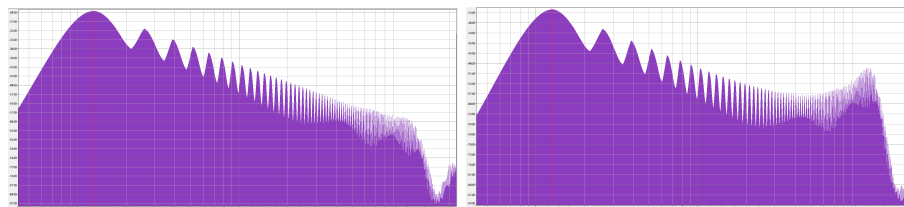


Figure 13: Nowe widmo dźwięku testowego

Na co należy spojrzeć w tych testach, to na wartości decybeli po lewej, oraz to jakim wartościom odpowiada zakres zaznaczony na czerwono. Widać, że na obrazku 13 wysokie częstotliwości mają głośność około -30 dB, podczas gdy na obrazku 12 mają około -35/-34 dB. **Widać, że filtr poprawnie zwiększa głośność podanego zakresu częstotliwości.**

2.2.3 Testy klasy LiveEQ

W zasadzie testy korektora sprawdzają jedynie, czy podane w konstruktorze zakresy częstotliwości są poprawnie modyfikowane. Warto też zauważyć, że częstotliwości często są podawane po podzieleniu na 2, ze względu na naturę mojej implementacji korektora (aczkolwiek nie ma to większego znaczenia).



(a) No Boost

(b) Boost High

(c) Boost Mid

(d) Boost Low

Figure 14: Testowanie zmiany zakresu częstotliwości przez LiveEQ

2.3 Zmiana głośności za pomocą Gain

Głośność Gain odpowiedzialna jest za zwiększanie/zmniejszanie głośności sygnału wchodzącego do wzmacniacza. Dla porównania, głośność Volume zmienia głośność sygnału wychodzącego ze wzmacniacza. Gain w porównaniu do Volume potrafi stanowić kluczową rolę w modyfikacji tonu dźwięku, ponieważ wykonuje modyfikacje jeszcze przed korekcją equalizerem, co potrafi oprócz zwiększenia głośności dodawać efekt "przesteru" dźwięku. Gain w tej implementacji jest stosowany po efektach, ale przed korektorem - taka decyzja implementacyjna wynikała właśnie z chęci nałożenia przesteru nawet na efekty, dzięki temu dźwięki wydają się większe i bardziej rezonujące:

Kawałek kodu funkcji processBuffer - kolejność przetwarzania dźwięku od góry do dołu

```
fLeft = flanger.processSample(fLeft);
fRight = flanger.processSample(fRight);

fLeft = delay.processSample(fLeft);
fRight = delay.processSample(fRight);

float[] s = reverb.processSample(fLeft, fRight);
fLeft = s[0];
fRight = s[1];

fLeft = gain.processSample(fLeft);
fRight = gain.processSample(fRight);

// Apply EQ
fLeft = liveEq.processSample(fLeft);
fRight = liveEq.processSample(fRight);
// Apply master volume
fLeft = volume.processSample(fLeft);
fRight = volume.processSample(fRight);
```

To co więc powinienem otrzymać, to podobne wyniki testów jak dla Volume, ale w tym miejscu spodziewać się jeszcze będzie można zmiany słyszalnego dźwięku:

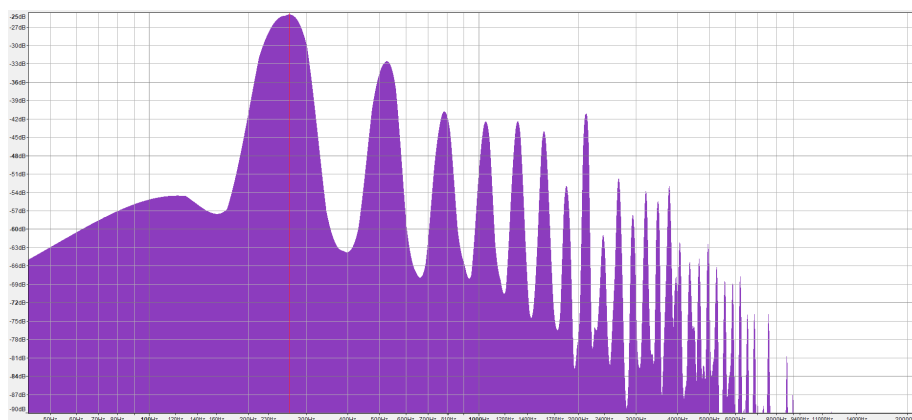


Figure 15: Widmo częstotliwościowe dźwięku testowego z Gainem 1.0

Znowu jako punkt odniesienia wezmę podobną częstotliwość fundamentalną jak w poprzednich testach. Aktualnym dźwiękiem testowym jest **pianoC4_nogain.wav**. Jak widać, głośność częstotliwości fundamentalnej to około -25 dB. Taką wartość przyjmuje dla uproszczenia obliczeń. Nowa wartość Gain zamiast 1.0 to będzie 4.0:

$$\Delta L = 20 \cdot \log_{10} \frac{v_n}{v_p}$$

$$\Delta L = 20 \cdot \log_{10} \frac{4.0}{1.0} \approx 20 \cdot 0.60205 = 12.041 [dB]$$

$$L_n = \Delta L + L_p \Rightarrow L_n \approx -25 + 12 = -13 [dB]$$

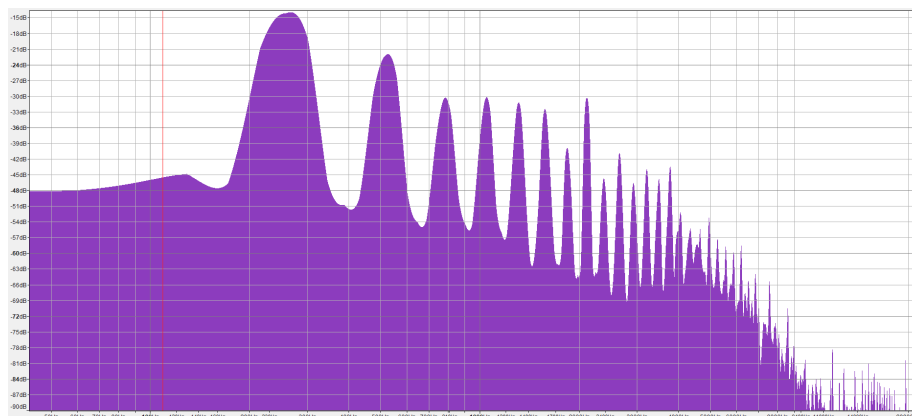


Figure 16: Widmo dźwięku testowego z Gainem 4.0

Z testu widać, że działanie Gaina również jest bardzo przewidywalnie. Tak jak przewidywałem obliczeniami, wartość głośności nowego dźwięku (pianoC4_4gain.wav) to mniej więcej 13 dB, co sprawia, że można stwierdzić o poprawnej funkcjonalności ustawienia Gain.

3 Testy funkcjonalności - efekty dźwiękowe

3.1 Klasa Delay

3.1.1 Krótki opis klasy Delay

Efekt Delay opiera się na implementacji cyklicznego bufora, który w każdej iteracji zapisuje na jakiejś nowej pozycji bufora, oraz zgodnie z wartością *rate* oraz *decay*, odczytuje z tego bufora na pozycji obliczonej dzięki właśnie tym dwóm parametrom. Ostatecznie, odczytany z bufora sample w reprezentacji float jest dodawany do aktualnie przetwarzanego sampla, pomnożony razy wartość *wet* - inaczej procent występowania efektu w wyjściowym dźwięku (w porównaniu z dźwiękiem niezmodyfikowanym).

3.1.2 Testowanie Delay

Testy Delay opierać się będą na sprawdzeniu jak bardzo zmienia dźwięk wartość *Decay*, oraz głównie, czy *Rate* tego efektu faktycznie ma znaczenie, oraz czy działanie tego parametru jest przewidywalne i zgodne z oczekiwaniami.

Na początku sprawdzę wartość *decay*. Ustawię *Wet* na 1.0, by jedynym sygnałem przechodzącym na wyjście było wyjście efektu. Dzięki temu będę mógł sprawdzać, czy kolejne dźwięki z efektu są faktycznie cichsze. Aktualnie jednak, pierwszy test wykonam na *Decay* = 1.0, czyli na ustawieniu, które teoretycznie powinno nieskończenie powtarzać ten dźwięk z ciągle takim samym poziomem głośności. Test rozpoczynam od parametrów Delaya na te powyżej, następnie zagram nutę C4, i będę włączać i wyłączać Delaya żeby zobaczyć, czy poprawnie działa (**pianoC4_decay10.wav**):



Figure 17: Waveform dźwięku testowego



Figure 18: Waveform dźwięku testowego z zaznaczonym podobieństwem biegunowości sygnałów

Z wyglądu waveformu wynika, że faktycznie - sygnał odgrywany jest za każdym razem z tą samą głośnością, czyli tak jak powinno być. Dla dodatkowego upewnienia się, przeanalizujemy pierwszy, faktycznie zagrany przez mnie dźwięk, jak i resztę powtórzonych przez Delay, i sprawdzimy czy znów występują podobieństwa:

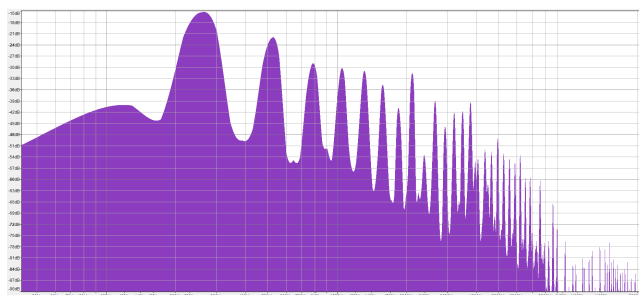


Figure 19: Widmo oryginalnego sygnału



Figure 20: Wzięty pod uwagę sygnał

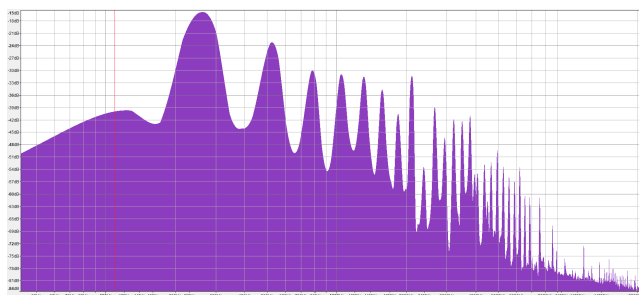


Figure 21: Widmo sygnału Delay



Figure 22: Sygnał delay, który analizuję

Jest w zasadzie idealnie. Występuje mała niedoskonałość, aczkolwiek może ona wynikać ze złego poboru kawałka dźwięku testowego, czy też zauważalnej lekkiej różnicy w waveformach sygnału input a sygnału Delay. Lecz różnica jest naprawdę niewielka - wręcz niezauważalna.

Teraz zmienię decay na 0.4, a rate na 1.0. Ten test będzie stanowić pewnego rodzaju weryfikację obu tych parametrów.

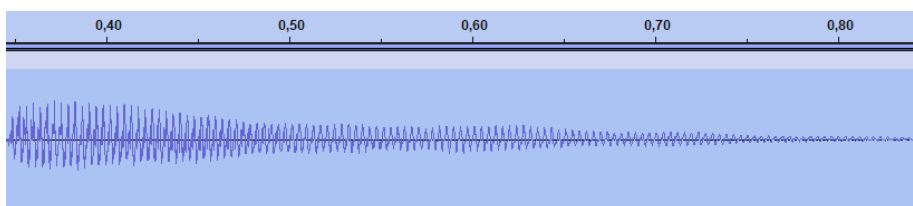


Figure 23: Czas od jednego sygnału do nowego 0.501 sekundy

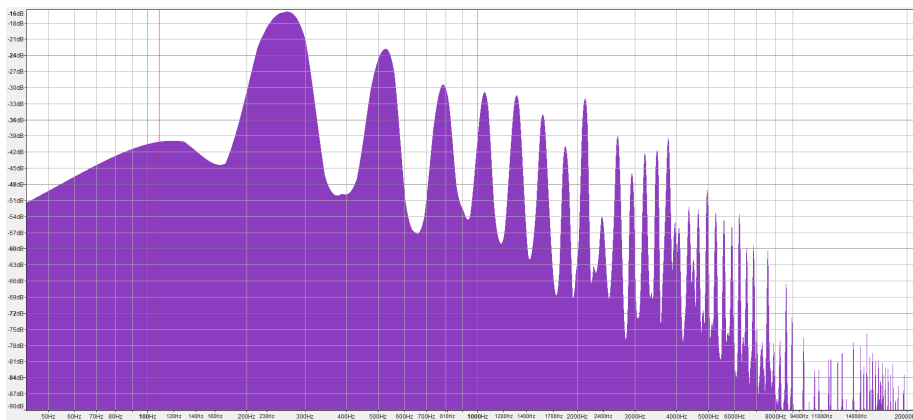


Figure 24: Widmo częstotliwości sygnału najgłośniejszego

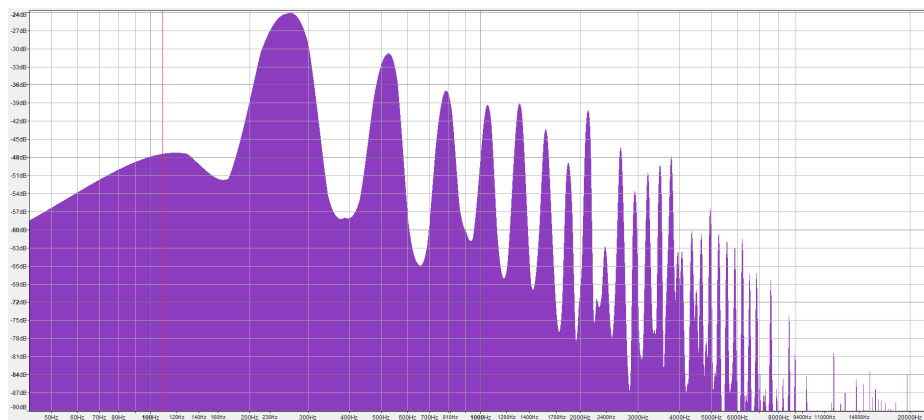


Figure 25: Widmo częstotliwości sygnału kolejnego po najgłośniejszym

$$\Delta L = 20 \cdot \log_{10} \frac{v_n}{v_p}$$

$$\Delta L = 20 \cdot \log_{10} \frac{0.4}{1.0} \approx 20 \cdot (-0.3979) = -7.9588 [dB]$$

$$\Delta L = L_n - L_p \Rightarrow L_n = \Delta L + L_p \Rightarrow L_n \approx -16 + (-8) = -24 [dB]$$

Wszystkie testy w zasadzie wychodzą poprawnie. Zwiększmy rate do 2.0 i zobaczmy, czy okres występowania sygnału faktycznie zwiększy się 2 razy:

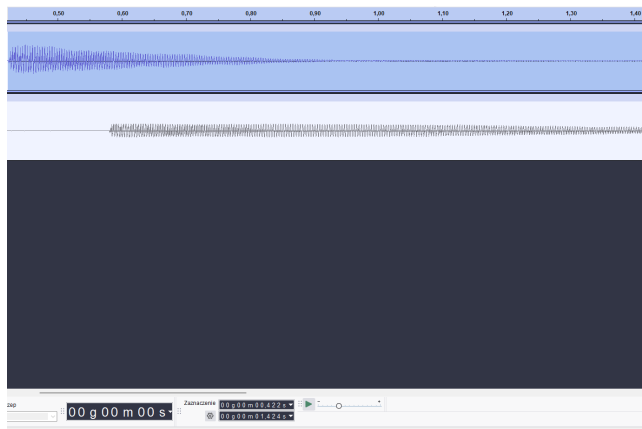


Figure 26: Nowy delay rate = 2.0

Odejmując widoczny na obrazku czas końca zaznaczenia od początku zaznaczenia, widać, że aktualnie pomiędzy sygnałami jest 1 sekunda i 2 setne odstępu. Wyniki testów są więc pozytywne.

3.2 Klasa Flanger

3.2.1 Krótki opis klasy Flanger

Flanger (podobnie jak Delay) posiada cykliczny bufor, na który w każdej iteracji zapisuje i z którego odczytuje. W zasadzie można to opisać jako "delay o bardzo krótkim rate i decay" - **działanie Flanger bowiem polega na odgrywaniu identycznego sygnału podczas sygnału oryginalnego, lecz z lekkim opóźnieniem, co tworzy bardzo ciekawy efekt.**

3.2.2 Testowanie Flanger

Testy Flangera przeprowadzę, sprawdzając działanie stosowania wartości Rate oraz na większym depth. **Depth pokaże, że flanger faktycznie funkcjonuje i mocno modyfikuje dźwięk, a Rate będzie odpowiedzialne za ustalenie częstotliwości modyfikacji dźwięku** (w nagraniu ta częstotliwość będzie dość mocno słyszalna ze względu na właśnie duży depth).



Figure 27: Ustawienia Flangera : wet 1, rate 1, depth = 2 (flangertest1.wav)

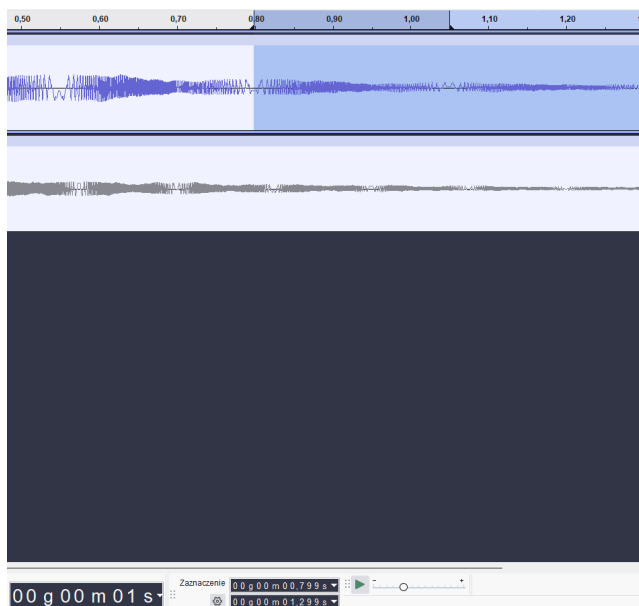


Figure 28: Waveform sygnału flangertest1.wav, długość pomiędzy powtórzeniami fazy = 0.5s

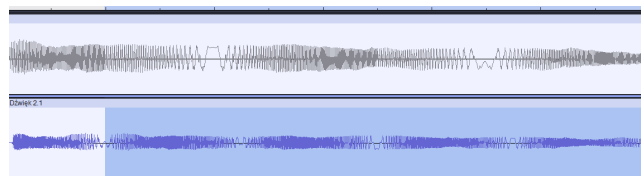


Figure 29: Waveform drugiego sygnału w porównaniu z drugim. Rate = 2, depth = 4.

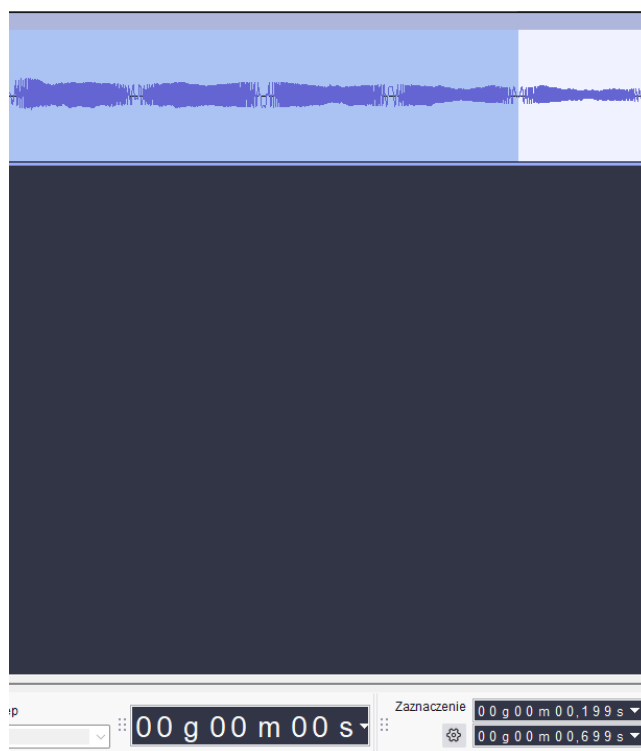


Figure 30: 2 razy większa częstotliwość modyfikacji dźwięku

Widać że pomiędzy podobnymi fazami Flanger faktycznir robi 2 razy więcej oscylacji. Zmianę depth słycać bardzo dobrze na wszystkich trzech nagraniach. Wyższy depth powoduje, że ton dźwięku bardziej się zmienia.

3.3 Klasa Reverb

3.3.1 Krótki opis klasy Reverb

Reverb zaimplementowany w tym zadaniu jest bardzo uproszczoną wersją tego efektu. Używa on czterech obiektów Delay z różnymi parametrami oraz z funkcją mieszającą sygnały wypuszczane z każdego Delaya za pomocą tzw. **Macierzy Hadamarda** wyglądającej w taki sposób (w przypadku czterech kanałów, czyli outputów z Delaya):

$$H = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

Ta macierz w zasadzie opisuje w jaki sposób mieszać wchodzące sygnały. Z rzędów tej macierzy tworzę wzory na 4 wymieszane, nowe sygnały:

```
float d1 = delayLeft.processSample(leftSample);
float d2 = delayRight.processSample(rightSample);
float d3 = delayLeft2.processSample(leftSample);
float d4 = delayRight2.processSample(rightSample);

float h1 = d1 + d2 + d3 + d4; // 1, 1, 1, 1
float h2 = d1 - d2 + d3 - d4; // 1, -1, 1, -1
float h3 = d1 + d2 - d3 - d4; // 1, 1, -1, -1
float h4 = d1 - d2 - d3 + d4; // 1, -1, -1, 1

float scale = 2;
h1 /= scale;
h2 /= scale;
h3 /= scale;
h4 /= scale;

float leftOutput = (h1 + h3) / (float) Math.sqrt(2);
float rightOutput = (h2 + 2 * h4) / (float) Math.sqrt(2);

leftOutput = leftOutput * wet + leftSample * (1 - wet);
rightOutput = rightOutput * wet + rightSample * (1 - wet);

return new float[]{leftOutput, rightOutput};
```

3.3.2 Testowanie Reverb

Stosując uproszczony algorytm Reverb, nie jestem w stanie wykonać sporej ilości testów. Z pewnością sprawdzę parametr Wet. Sprawdzę też, czy parametr length/decay faktycznie zmienia "długość" odgrywania efektu reverbu.

W zasadzie testu Wet nie mogę pokazać inaczej niż na nagraniu. Polecam wysłuchać pliku audio **reverb_wettest1.wav**.

Z kolei przy teście length delaya, można zarówno posłuchać rezultatu, jak i zobaczyć, że waveforms kolejnych sygnałów robią się faktycznie dłuższe - czyli inaczej, dłużej rezonują.



Figure 31: Wygląd waveform dla testowego dźwięku - Reverb